

```
In [12]: import pandas as pd
```

```
In [13]: # Define max_rows para 9999
pd.set_option('display.max_rows', 9999)
```

```
In [14]: # Criar uma variável
dados_csv = None

# Ler o conteúdo do arquivo CSV
dados_csv = pd.read_csv(
    './data/picoweb.csv', # caminho do arquivo
    sep=';',              # separador de colunas
    engine='python',      # engine
    encoding='utf-8'      # encoding
)
```

```
In [15]: print("=== INFORMAÇÕES GERAIS DO DATASET ===")
print("="*65)
print(dados_csv.describe())
print("="*65 + "\n")
```

```
=== INFORMAÇÕES GERAIS DO DATASET ===
=====
      ID      Duration      Pulse      Maxpulse      Calories
count  32.000000    32.000000    32.000000    32.000000    30.000000
mean   15.500000    68.437500   103.500000   128.500000   3046.800000
std     9.380832    70.039591    7.832933    12.998759    660.037794
min     0.000000    30.000000    90.000000   101.000000   1951.000000
25%     7.750000    60.000000   100.000000   120.000000   2507.000000
50%    15.500000    60.000000   102.500000   127.500000   2912.000000
75%    23.250000    60.000000   106.500000   132.250000   3439.750000
max    31.000000   450.000000   130.000000   175.000000   4790.000000
=====
```

```
In [16]: # Informações completas do dataset
print("=== INFORMAÇÕES COMPLETAS DO DATASET ===")
print("="*65)
print(dados_csv.info())
print("="*65)
```

```
=== INFORMAÇÕES COMPLETAS DO DATASET ===
=====
<class 'pandas.DataFrame'>
RangeIndex: 32 entries, 0 to 31
Data columns (total 6 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   ID          32 non-null    int64
 1   Duration    32 non-null    int64
 2   Date        31 non-null    str
 3   Pulse       32 non-null    int64
 4   Maxpulse    32 non-null    int64
 5   Calories    30 non-null    float64
dtypes: float64(1), int64(4), str(1)
memory usage: 1.6 KB
None
=====
```

```
In [19]: # TOTAL DE LINHAS
print("\n" + "="*65)
print("1. TOTAL DE LINHAS:")
print(f"Total de linhas de dados: {len(dados_csv) - 1}")
print(f"Total de linhas no arquivo: {len(dados_csv)}")
print("="*65)
```

```
=====
1. TOTAL DE LINHAS:
Total de linhas de dados: 31
Total de linhas no arquivo: 32
=====
```

```
In [8]: # TOTAL DE COLUNAS
print("\n" + "="*65)
print("2 TOTAL DE COLUNAS:")
print(f"O dataset possui {len(dados_csv.columns)} colunas")
print(f"Colunas: {dados_csv.columns.tolist()}")
print("="*65)
```

```
=====
2 TOTAL DE COLUNAS:
O dataset possui 6 colunas
Colunas: ['ID', 'Duration', 'Date', 'Pulse', 'Maxpulse', 'Calories']
=====
```

```
In [9]: # QUANTIDADE DE DADOS NULOS
print("\n" + "="*65)
print("3. DADOS NULOS POR COLUNA:")
nulos_por_coluna = dados_csv.isnull().sum()
print("Nulos por coluna:")
print(nulos_por_coluna)
print(f"\nTOTAL de valores nulos no dataset: {dados_csv.isnull().sum().sum()}")
print("="*65)
```

```
=====
3. DADOS NULOS POR COLUNA:
Nulos por coluna:
ID          0
Duration    0
Date        1
Pulse       0
Maxpulse    0
Calories    2
dtype: int64

TOTAL de valores nulos no dataset: 3
=====
```

```
In [10]: # TIPO DE DADO DE CADA COLUNA
print("\n" + "="*65)
print("4. TIPO DE DADO POR COLUNA:")
print(dados_csv.dtypes)

# Explicação detalhada dos tipos
print("\nDetalhamento dos tipos encontrados:")
for coluna in dados_csv.columns:
    tipo = dados_csv[coluna].dtype
    if tipo == 'int64':
        print(f" - {coluna}: Inteiro (int64) - valores inteiros")
    elif tipo == 'float64':
```

```

        print(f" - {coluna}: Decimal/Ponto flutuante (float64) - permite valores decimais e NaN")
    elif tipo == 'object':
        print(f" - {coluna}: Texto/String (object) - valores alfanuméricos")
print("="*65)

```

4. TIPO DE DADO POR COLUNA:

```

ID                int64
Duration          int64
Date              str
Pulse             int64
Maxpulse          int64
Calories          float64
dtype: object

```

Detalhamento dos tipos encontrados:

- ID: Inteiro (int64) - valores inteiros
- Duration: Inteiro (int64) - valores inteiros
- Pulse: Inteiro (int64) - valores inteiros
- Maxpulse: Inteiro (int64) - valores inteiros
- Calories: Decimal/Ponto flutuante (float64) - permite valores decimais e NaN

```

In [11]: # QUANTIDADE DE MEMÓRIA UTILIZADA
print("\n" + "="*65)
print("5. MEMÓRIA UTILIZADA:")

# Memória detalhada por coluna
memoria_detalhada = dados_csv.memory_usage(deep=True)
print("Memória por coluna:")
for coluna in dados_csv.columns:
    memoria_bytes = memoria_detalhada[coluna]
    memoria_kb = memoria_bytes / 1024
    print(f" - {coluna}: {memoria_bytes:>10,} bytes ({memoria_kb:.2f} KB)")

# Memória total
memoria_total_bytes = memoria_detalhada.sum()
memoria_total_kb = memoria_total_bytes / 1024
memoria_total_mb = memoria_total_kb / 1024

print(f"\nMEMÓRIA TOTAL:")
print(f" - Bytes: {memoria_total_bytes:,} bytes")
print(f" - Kilobytes (KB): {memoria_total_kb:.2f} KB")
print(f" - Megabytes (MB): {memoria_total_mb:.4f} MB")
print("="*65)

```

```
=====
5. MEMÓRIA UTILIZADA:
Memória por coluna:
- ID:          256 bytes (0.25 KB)
- Duration:    256 bytes (0.25 KB)
- Date:        1,919 bytes (1.87 KB)
- Pulse:       256 bytes (0.25 KB)
- Maxpulse:    256 bytes (0.25 KB)
- Calories:    256 bytes (0.25 KB)

MEMÓRIA TOTAL:
- Bytes: 3,331 bytes
- Kilobytes (KB): 3.25 KB
- Megabytes (MB): 0.0032 MB
=====
```

In []: